Campus Lost-and-Found Portal

Project Report — Group 23

A PHP and PostgreSQL web application with an AJAX-driven interface

Repository: github.com/soumya-choudhary/Lost-and-Found | Group 23 | November 2026

1. Abstract

The Campus Lost-and-Found Portal is a **multi-tier web application** through which campus users report lost or found items, search a public catalog, submit **ownership claims** supported by evidence, and follow each item from report to resolution. The presentation tier uses HTML5, CSS3, Bootstrap 5.3.3, JavaScript and jQuery 3.7.1; the application tier is written in PHP; the data tier is a PostgreSQL database accessed through PDO. Catalog search, status changes, claim submission and administrative actions are issued as jQuery AJAX requests to JSON endpoints, so pages update without a full reload. Access is governed by two roles, *user* and *admin*, enforced by middleware and by server-side ownership checks. Data is held in **five relational tables** protected by CHECK, UNIQUE and foreign-key constraints, and every significant item event is recorded in an **activity log**.

Keywords: *lost and found, PHP, PostgreSQL, PDO, jQuery, AJAX, role-based access control, ownership claims.*

2. Problem Statement

A campus lost-and-found process requires a **shared record** that can be searched and filtered, a means of establishing that a claimant is the **genuine owner** before an item is released, and a **visible status** for each item from report to resolution. Without a structured record there is no uniform way to search items, to compare a claim with supporting evidence, or to trace who acted on an item and when. This project addresses these needs with a web portal that stores every report and claim in a relational database, exposes them through a live-filtered catalog, and records the lifecycle of each item.

3. Project Objectives

Table 1 lists the eight objectives and the section that realises each.

Table 1. *Project objectives and the sections that realise them.*

No.	Objective	Section
O1	Provide a public catalog searchable by keyword and filterable by type, category and status.	6.2
O2	Update results, statuses and claim decisions without full page reloads.	6.2, 6.4, 7
O3	Allow items to be reported with an optional photograph, including by visitors who are not signed in.	6.3
O4	Verify ownership through evidence-based claims that the item's own reporter cannot file.	6.4
O5	Let the reporter or an administrator decide claims and manage item status.	6.4, 7
O6	Keep an audit trail of item events.	5, 6.4
O7	Provide a student area and an administrator console for claims, items and users.	4.2, 6.5
O8	Protect credentials, sessions, queries and output.	6.6

4. System Architecture and Technology Stack

4.1 Architecture and technology stack

The system follows a three-tier layered architecture. The **presentation tier** consists of the PHP page scripts, the shared *layouts/* files and the assets in *public/assets/*. The **application tier** consists of the controllers and middleware in *app/*, the JSON endpoints in *api/* and the helper functions in *config/app.php*. The **data tier** consists of the repository classes, the PDO singleton in *config/database.php* and the PostgreSQL database. Table 2 lists the technologies used in each layer.

Table 2. *Technology stack.*

Layer	Technology	Role in the system
Presentation	HTML5, CSS3, JavaScript, jQuery 3.7.1, Bootstrap 5.3.3, Font Awesome 6.5.1	Server-rendered pages with a responsive Bootstrap layout; jQuery in items.js, admin.js and auth.js (AJAX in items.js and admin.js); styles in style.css. Bootstrap, Font Awesome and jQuery are loaded from CDNs in the layout files.
Application	PHP 8 (uses match and str_starts_with)	Page scripts, controllers, middleware, repositories, JSON endpoints and session handling.
Database	PostgreSQL through PDO (pdo_pgsql and pgsql extensions)	Database lost_and_found_db created from database/schema.sql: five relational tables with constraints, foreign keys and indexes.
Web server	Apache under XAMPP (as documented in README.md)	README: project in htdocs/Lost-and-Found, PostgreSQL installed separately (XAMPP MySQL unused); served at APP_URL (default http://localhost/Lost-and-Found).
Configuration	.env read by loadEnv() in config/database.php	DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASS, APP_NAME, APP_ENV, APP_URL; template in .env.example; .env and uploads/ are excluded by .gitignore.

4.2 Web pages

Table 3 lists every web page, the visitors who may open it and its behaviour; access levels are enforced by *AuthMiddleware* and *AdminMiddleware* (Section 6.1).

Table 3. *Web pages, access levels and behaviour.*

Page	Access	Purpose and behaviour
index.php (alias home.php)	Public	Catalog: lost / found / resolved counters, the 60 newest items as cards, filter bar that queries api/items.php.
login.php	Guests only	Sign-in form with Student and Admin demo buttons; redirects by role.
register.php	Guests only	Account creation with live password-match feedback.
logout.php	Any visitor	Ends the session and redirects to the login page.
report-item.php	Public	Lost or found report form with an optional photo.
item-details.php	Public; parts limited by role	Item record and reporter contact; status buttons and received claims (reporter or admin only); claim modal (everyone except the reporter); activity log.
student/dashboard.php	Signed in	Counts of reported items, submitted claims and resolved items; the four most recent reports.
student/my-reports.php	Signed in	The user's reported items with a claim count linking to the item page.
student/my-claims.php	Signed in	Claims filed with the user's email: status, proof, resolution notes.
admin/dashboard.php	Admin	Cards for lost, found, pending claims and resolved items; the five most recent claims.
admin/claims.php	Admin	Claims desk (up to 100): status filter buttons and a review modal with note, Approve and Reject.
admin/items.php	Admin	Inventory (up to 200 items): text filter, status dropdown and Delete with confirmation.
admin/users.php	Admin	Directory (up to 100 users) with a role dropdown; the admin's own row shows "Current Session".

4.3 PHP scripts and supporting files

Table 4 names the PHP scripts outside the page directories, the methods of each class and the supporting files.

Table 4. *PHP scripts, class methods and supporting files.*

Location	Scripts and their responsibility
api/	items.php, update-status.php, claims.php, admin-claim-action.php, admin-item-delete.php, admin-user-role.php (JSON endpoints, Section 7); claim-action.php only includes admin-claim-action.php.
app/controllers/	**AuthController.php:** handleLogin(), handleRegister(), logout(). **ItemController.php:** getCategories(), handleReportSubmission(), handleAjaxFilter(), renderItemCard(), getItem().
app/repositories/	**ItemRepository.php:** getAllCategories(), create(), searchAndFilter(), findById(), updateStatus(), getByUserId(), getStats(), delete(), logActivity(), getActivityLogs(). **ClaimRepository.php:** create(), findById(), getByItemId(), getByUserEmail(), getAll(), getPendingCount(), updateClaimStatus(). **UserRepository.php:** findByEmail(), findById(), emailExists(), create(), getAll(), updateRole().
app/middleware/	**AuthMiddleware.php:** check(), guestOnly(). **AdminMiddleware.php:** check().
config/	**app.php:** session start, APP_NAME, APP_URL and helpers url(), redirect(), setFlash(), getFlash(), isLoggedIn(), isAdmin(), currentUser(), sanitize(), isItemOwner(), canManageItem(), typeBadgeClass(), statusBadgeClass(), claimBadgeClass(), itemImageUrl(). **database.php:** loadEnv() and Database::getConnection() (PDO singleton). **db.php:** wrapper that includes database.php.
layouts/	**header.php:** Bootstrap navbar and dismissible flash alert. **footer.php:** loads jQuery, Bootstrap, auth.js and the page script set in $extraScripts.
Other files	database/schema.sql; public/assets/css/style.css, js/items.js, js/admin.js, js/auth.js, img/placeholder.svg; README.md, LICENSE (MIT), .env.example, .gitignore.

5. Database Design

The script *database/schema.sql* drops any existing tables (*DROP TABLE IF EXISTS ... CASCADE*) and creates five tables in the database **lost_and_found_db**. Table 5 gives the type, nullability and constraints of every column, and Table 6 explains the foreign-key rules.

Table 5. *Table definitions (database/schema.sql).*

Table	Column	Type	Constraints and defaults
users	id	SERIAL	PRIMARY KEY
	name	VARCHAR(100)	NOT NULL
	email	VARCHAR(150)	UNIQUE, NOT NULL
	password_hash	VARCHAR(255)	NOT NULL
	phone	VARCHAR(20)	nullable
	role	VARCHAR(20)	NOT NULL, default 'user', CHECK in ('user','admin')
	created_at	TIMESTAMP WITH TIME ZONE	default CURRENT_TIMESTAMP

Table	Column	Type	Constraints and defaults
categories	id	SERIAL	PRIMARY KEY
	name	VARCHAR(60)	UNIQUE, NOT NULL
	icon	VARCHAR(60)	default 'fa-box'
	description	VARCHAR(255)	nullable
items	id	SERIAL	PRIMARY KEY
	item_type	VARCHAR(10)	NOT NULL, CHECK in ('lost','found')
	title	VARCHAR(150)	NOT NULL
	category_id	INT	NOT NULL, FK to categories(id) ON DELETE RESTRICT
	location	VARCHAR(150)	NOT NULL
	event_date	DATE	NOT NULL
	description	TEXT	NOT NULL
	image_url	VARCHAR(255)	nullable
	status	VARCHAR(20)	NOT NULL, default 'Open', CHECK in ('Open','Claim Requested','Resolved')
	user_id	INT	nullable, FK to users(id) ON DELETE SET NULL
	contact_name	VARCHAR(100)	NOT NULL
	contact_email	VARCHAR(150)	NOT NULL
	contact_phone	VARCHAR(20)	nullable
	created_at	TIMESTAMP WITH TIME ZONE	default CURRENT_TIMESTAMP
	updated_at	TIMESTAMP WITH TIME ZONE	default CURRENT_TIMESTAMP
claims	id	SERIAL	PRIMARY KEY
	item_id	INT	NOT NULL, FK to items(id) ON DELETE CASCADE
	claimant_name	VARCHAR(100)	NOT NULL
	claimant_email	VARCHAR(150)	NOT NULL
	claimant_phone	VARCHAR(20)	nullable
	proof_description	TEXT	NOT NULL
	proof_image_url	VARCHAR(255)	nullable
	status	VARCHAR(20)	NOT NULL, default 'Pending', CHECK in ('Pending','Approved','Rejected')
	admin_notes	TEXT	nullable
	created_at	TIMESTAMP WITH TIME ZONE	default CURRENT_TIMESTAMP
	resolved_at	TIMESTAMP WITH TIME ZONE	nullable
activity_logs	id	SERIAL	PRIMARY KEY
	item_id	INT	nullable, FK to items(id) ON DELETE CASCADE
	action	VARCHAR(100)	NOT NULL
	details	TEXT	nullable
	created_at	TIMESTAMP WITH TIME ZONE	default CURRENT_TIMESTAMP

Table 6. *Foreign-key behaviour.*

Relationship	Rule	Effect
items.category_id to categories.id	ON DELETE RESTRICT	A category that items refer to cannot be deleted.
items.user_id to users.id	ON DELETE SET NULL	Deleting a user keeps the item; the reporter link becomes NULL and the contact fields remain.
claims.item_id to items.id	ON DELETE CASCADE	Deleting an item (api/admin-item-delete.php) removes its claims.
activity_logs.item_id to items.id	ON DELETE CASCADE	Deleting an item removes its log entries.

Six indexes support the queries: idx_items_type_status (item_type, status), idx_items_category (category_id), idx_items_event_date (event_date), idx_items_title_location (title, location), idx_claims_item (item_id) and idx_claims_status (status). The script seeds eight categories (Electronics; Student IDs & Cards; Wallets & Purses; Keys & Keychains; Books & Stationery; Bags & Backpacks; Clothing & Accessories; Others), two users, Campus Admin (*admin@campus.edu*, role admin) and Demo Student (*student@campus.edu*, role user), which share one bcrypt hash of the password *admin123*, and four sample items (two lost, two found) with statuses Open, Open, Claim Requested and Resolved.

6. Functional Modules

The implemented work comprises the **database layer** (Section 5), **authentication**, the **public catalog**, **item reporting**, **item management with ownership claims**, the **student portal**, the **administrator console**, the shared services (*config/app.php*, *layouts/*, *style.css*) and the project files *README.md*, *LICENSE*, *.env.example* and *.gitignore*.

6.1 Authentication and session management

Registration (*register.php*, *AuthController::handleRegister()*) requires a name of at least two characters, a valid email address that is not yet registered (compared case-insensitively) and a password of at least six characters with a matching confirmation; the phone number is optional. The password is hashed with *password_hash(..., PASSWORD_BCRYPT)*, the account receives the role *user*, and the user is signed in and redirected to *student/dashboard.php*.

Login and logout. *handleLogin()* validates the email format and a non-empty password, retrieves the user by email, checks the password with *password_verify*, calls *session_regenerate_id(true)*, stores user_id, user_name, user_email and user_role in the session, and redirects administrators to *admin/dashboard.php* and other users to *student/dashboard.php*; every failure produces the single message "Invalid email or password". *logout()* clears the session array, expires the session cookie, destroys the session and redirects to the login page with a flash message.

Route guards. *AuthMiddleware::check()* redirects guests to the login page, *guestOnly()* redirects signed-in users away from login and registration, and *AdminMiddleware::check()* sends guests to login and non-admins to *student/dashboard.php* with an error message. **Client-side support.** In *auth.js*, *checkPasswordMatch()* gives live feedback and an alert blocks a password shorter than six characters or a mismatch; a *.fill-demo-btn* handler fills the login form with the demo credentials; a *.toggle-password-btn* handler switches a password field between password and text (no page currently contains such a button).

6.2 Public catalog and live search

The counters of lost, found and resolved items come from *ItemRepository::getStats()*. The filter bar offers a keyword box (matched against title, description and location with PostgreSQL *ILIKE*), All / Lost / Found radio buttons, category and status dropdowns and a reset button. After a 200 ms debounce, *filterItems()* sends a jQuery *$.getJSON* request to *api/items.php*; the JSON reply replaces the card grid and the "Showing N items" counter while a loading indicator is displayed. Cards are rendered by *ItemController::renderItemCard()*, and a delegated click handler on *.item-card* opens the card's *data-href* unless a link or button inside the card was clicked.

6.3 Item reporting

The report form (*report-item.php*) is open to every visitor; the report type defaults from the *?type=* query string, contact name and email are pre-filled for signed-in users, and the date field has today as its maximum. *ItemController::handleReportSubmission()* validates that the type is lost or found, the title has at least three characters, a category is selected, the location is present, the date is present and not in the future, the description has at least five characters, the contact name is present and the contact email is valid; errors are listed and the form is re-populated. An optional photograph must be a jpg, jpeg, png or webp file of at most 5 MB and is saved in *uploads/items/* under a name built from the timestamp and *random_bytes()*; its path is stored in *items.image_url*. *ItemRepository::create()* then inserts the item with status Open (user_id from the session, or NULL for a guest) and logs "Created", and the user is redirected to *item-details.php?id=N*.

6.4 Item details, status control and ownership claims

Ownership rules. *isItemOwner()* is true when the item's user_id equals the session user's id or its contact_email equals the user's email (case-insensitive); *canManageItem()* is true for an administrator or the owner. Only such users see the status buttons (Open, In Claim, Resolved), which post to *api/update-status.php* and update the badge in place, and the list of received claims. The activity log displays the ten most recent entries.

Claim submission. A modal is offered to every visitor except the reporter while the item is not Resolved; it collects name, campus email, an optional phone number, a proof description of at least five characters and an optional receipt or photograph, and is submitted with jQuery *FormData*. *api/claims.php* rejects the reporter and stores an optional proof image (jpg, jpeg, png, webp) in *uploads/claims/*. *ClaimRepository::create()* inserts the claim as Pending, changes the item from Open to Claim Requested (only when its status is Open) and logs "Claim Submitted" within one transaction, rolling back on error.

Claim decisions. The reporter or an administrator sees each received claim with a link to any proof photograph and, for Pending claims, Reject and Accept buttons (*.user-claim-decision-btn*); a browser prompt collects an optional note before *api/admin-claim-action.php* is called. *updateClaimStatus()* is likewise transactional: it stores the decision, the note and *resolved_at*, and an Approved claim sets the item to Resolved and logs "Claim Approved".

6.5 Student portal and administrator console

The student pages are guarded by *AuthMiddleware* and the administrator pages by *AdminMiddleware*; their content is summarised in Table 3. On the claims desk the status filter buttons work on the client and a row updates in place after a decision; the inventory filter hides table rows that do not contain the typed text. After each successful administrator action, *admin.js* calls *notify()*, which shows a green message for three seconds (fade in, 3000 ms delay, fade out).

6.6 Security mechanisms

Table 7 summarises the mechanisms that protect credentials, queries, output, sessions and uploaded files, with the code that implements each.

Table 7. *Security mechanisms and their location in the code.*

Mechanism	Implementation	Location
Password storage	password_hash with PASSWORD_BCRYPT; password_verify at login.	AuthController, UserRepository::create()
SQL injection	Every repository query that takes input is a PDO prepared statement with bound parameters; native prepares (ATTR_EMULATE_PREPARES false) and ERRMODE_EXCEPTION. Fixed queries without input (category list, statistics, pending count) use query(); the catalog LIMIT is clamped to an integer from 1 to 200.	app/repositories/, config/database.php
Cross-site scripting	Output is escaped by sanitize(), a wrapper around htmlspecialchars with ENT_QUOTES and UTF-8.	config/app.php
Session security	session_regenerate_id(true) at login and registration; logout clears the array, expires the cookie and destroys the session.	AuthController
Authorization	Role middleware for student and admin pages; server-side checks in the endpoints (admin or reporter for status and claim decisions; admin only for deletion and roles).	app/middleware/, config/app.php, api/
Integrity rules	Reporters cannot claim their own items; an admin cannot demote their own account.	api/claims.php, api/admin-user-role.php
Upload control	Extension whitelist (jpg, jpeg, png, webp), 5 MB limit for item photos, random file names; uploads/ and .env excluded from version control.	ItemController, api/claims.php, .gitignore
Data integrity	CHECK, UNIQUE and foreign-key constraints; transactions around claim creation and decisions; activity_logs actions "Created", "Status Updated", "Claim Submitted", "Claim Approved".	schema.sql, ClaimRepository, ItemRepository

7. API / Endpoints Dictionary

Table 8 documents the six JSON endpoints used by the AJAX interface. The POST endpoints report validation, authorization and database failures as JSON with *success* set to false and an explanatory *message*.

Table 8. *API endpoints.*

Endpoint	Method and access	Request parameters	Response and behaviour
api/items.php	GET; public	q, type, category, status, sort (newest, oldest, date_desc)	JSON {success, count, html}; ILIKE search; limit 1 to 200 (default 100); calls ItemController::handleAjaxFilter().
api/update-status.php	POST; signed in; admin or item reporter	item_id, status (Open / Claim Requested / Resolved), remarks (optional)	JSON {success, item_id, new_status, message}; logs "Status Updated" as "Changed by NAME (Admin or Reporter). remarks".
api/claims.php	POST; any visitor except the item's reporter	item_id, claimant_name, claimant_email, claimant_phone (optional), proof_description, proof_image (optional file)	JSON {success, claim_id, message}; claim saved as Pending; item moves from Open to Claim Requested.
api/admin-claim-action.php (alias claim-action.php)	POST; signed in; admin or item reporter	claim_id, decision (Approved / Rejected), admin_notes	JSON {success, claim_id, decision, message}; note stored as "Decided by NAME (Admin or Reporter): notes"; Approved sets the item to Resolved.
api/admin-item-delete.php	POST; admin only	item_id	JSON {success, item_id, message}; claims and logs removed by ON DELETE CASCADE.
api/admin-user-role.php	POST; admin only	user_id, role (user / admin)	JSON {success, user_id, new_role, message}; an admin's attempt to demote their own account is refused.

8. Conclusion

The Campus Lost-and-Found Portal implements the **complete lost-and-found cycle** with PHP and PostgreSQL behind an HTML, CSS, Bootstrap and jQuery interface: role-based registration and login, a public catalog with live AJAX search, item reporting with photo upload, **evidence-based ownership claims** decided by the reporter or an administrator, status tracking with an **audit trail**, student pages and an administrator console. Pages, controllers, middleware, repositories and JSON endpoints sit over a five-table relational schema whose constraints, foreign-key rules and transactions keep item, claim and log data consistent.